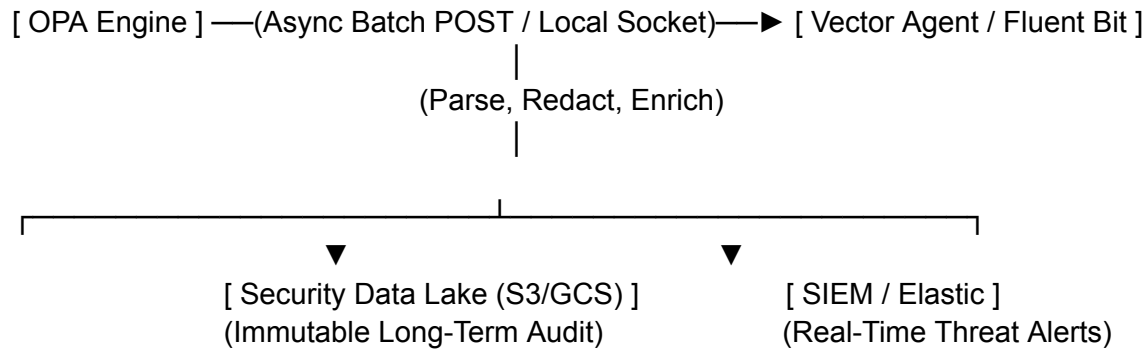


An asynchronous decision logging pipeline decouples policy enforcement from audit storage, ensuring that high-throughput agent tool invocations suffer zero latency penalties from log IO. Open Policy Agent (OPA) buffers decision events locally and pushes them in batches to a log collector (**Vector** or **Fluent Bit**), which parses, redacts, enriches, and routes the audit logs to a security data lake or SIEM.

## 1. Architecture & Data Flow



## 2. OPA Sidecar Decision Logging Setup

Configure OPA's `decision_logs` plugin in `/etc/opa/config.yaml` to buffer events in memory and asynchronously flush them to Vector's HTTP ingress listener:

YAML

```
# /etc/opa/config.yaml
decision_logs:
  service: vector_collector
  path: /v1/opa/decisions
  reporting:
    min_delay_seconds: 2
    max_delay_seconds: 5
    buffer_limit_bytes: 10485760 # 10MB memory buffer before dropping
```

services:

```
vector_collector:
  url: http://127.0.0.1:8686
  headers:
    Authorization: "Bearer ${VECTOR_INGRESS_TOKEN}"
```

## 3. Vector Collector Configuration

Vector acts as the local log aggregator on the node. It receives OPA's raw decision arrays, flattens them into individual records, enriches them with host/agent metadata, redacts sensitive payload attributes, and ships them to long-term storage.

Ini, TOML

```
# /etc/vector/vector.toml
```

```
[sources.opa_decisions]  
type = "http_server"  
address = "127.0.0.1:8686"  
path = "/v1/opa/decisions"
```

```
[transforms.parse_and_flatten]  
type = "remap"  
inputs = ["opa_decisions"]  
source = ""  
# 1. Parse payload array from OPA batch POST  
. = unnest!(.labels)
```

```
# 2. Extract decision metadata  
.decision_id = .input.decision_id  
.spiffe_id = .input.input.identity.spiffe_id  
.tool_name = .input.input.request.tool_name  
.allowed = .input.result.allow  
.execution_depth = .input.input.request.execution_depth
```

```
# 3. Redact sensitive input parameters (PII/Tokens)  
if exists(.input.input.request.parameters.api_key) {  
  .input.input.request.parameters.api_key = "[REDACTED]"  
}
```

```
# 4. Add node and cluster metadata  
.cluster_id = get_env_var("CLUSTER_ID") or "unknown"  
.node_id = get_env_var("NODE_ID") or "unknown"  
.event_timestamp = now()  
""
```

```
# Sink 1: Immutable storage in AWS S3 (Parquet format for cheap query analysis)
```

```
[sinks.s3_audit_lake]  
type = "aws_s3"  
inputs = ["parse_and_flatten"]  
bucket = "agent-security-audit-lake"  
key_prefix = "opa-decisions/year=%Y/month=%m/day=%d/"  
encoding.codec = "json"  
compression = "gzip"  
buffer.max_events = 5000  
buffer.when_full = "block"
```

```
# Sink 2: Real-time SIEM for anomaly detection
```

```
[sinks.elasticsearch_siem]
type = "elasticsearch"
inputs = ["parse_and_flatten"]
endpoints = ["https://siem.swarm.internal:9200"]
index = "opa-decision-logs-%Y.%m.%d"
auth.strategy = "bearer"
auth.token = "${ES_API_TOKEN}"
```

## 4. Security Auditing & SIEM Detection Rules

Once OPA decision logs land in the SIEM, apply real-time detection rules to flag rogue agent behavior:

| Threat Pattern                           | SIEM Query / Rule                                                                                                                              | Risk Level      | Triggered Response                                                     |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|------------------------------------------------------------------------|
| <b>Tool Enumeration Attack</b>           | <code>allowed == false</code><br>aggregated by <code>spiffe_id</code><br>count <code>\$ &gt; 10\$</code> in<br><code>\$1\, \text{min}\$</code> | <b>HIGH</b>     | Temporarily trip the circuit breaker for the specific agent SPIFFE ID. |
| <b>Max Depth Violation</b>               | <code>.execution_depth &gt; 3</code><br>AND <code>allowed == false</code>                                                                      | <b>MEDIUM</b>   | Alert SOC of potential recursive delegation explosion or logic loop.   |
| <b>Unauthorized Privilege Escalation</b> | <code>spiffe_id:</code><br><code>"*data-analyst*"</code> AND<br><code>tool_name:</code><br><code>"restart_service"</code>                      | <b>CRITICAL</b> | Revoke agent SVID via SPIRE API and alert security team immediately.   |

Storing and analyzing high-volume OPA decision logs requires a schema optimized for columnar compression, fast time-series partitioning, and low-latency point lookups on agent identities (`spiffe_id`). ClickHouse handles millions of ingested log rows per second when using column-level dictionary encoding (`LowCardinality`) and targeted skip indexes.

## 1. Optimized ClickHouse Table Schema

SQL

```
CREATE TABLE security_audit.opa_decision_logs
(
    -- Time-Series Ordering
    event_time      DateTime64(3, 'UTC') CODEC(DoubleDelta, ZSTD(1)),

    -- Identifiers
    decision_id     UUID CODEC(ZSTD(1)),
    spiffe_id       String CODEC(ZSTD(3)),
    trust_domain     LowCardinality(String) CODEC(ZSTD(1)),

    -- Decision Context
    allowed         UInt8 CODEC(T64, ZSTD(1)), -- 1 = true, 0 = false
    tool_name       LowCardinality(String) CODEC(ZSTD(1)),
    action          LowCardinality(String) CODEC(ZSTD(1)),
    target_resource  String CODEC(ZSTD(3)),
    execution_depth  UInt8 CODEC(T64, ZSTD(1)),

    -- Fleet Metadata
    cluster_id      LowCardinality(String) CODEC(ZSTD(1)),
    node_id         LowCardinality(String) CODEC(ZSTD(1)),

    -- Dynamic Payload & Parameters (Raw Context)
    parameters      Map(String, String) CODEC(ZSTD(1)),
    raw_input       String CODEC(ZSTD(6)), -- Compressed raw JSON for forensics

    -- Skip Indexes for Fast Filtering
    INDEX idx_spiffe spiffe_id TYPE bloom_filter(0.01) GRANULARITY 1,
    INDEX idx_resource target_resource TYPE tokenbf_v1(30720, 2, 0) GRANULARITY 1
)
ENGINE = MergeTree()
PARTITION BY toYYYYMM(event_time)
ORDER BY (event_time, allowed, tool_name, spiffe_id)
TTL event_time + INTERVAL 90 DAY DELETE;
```

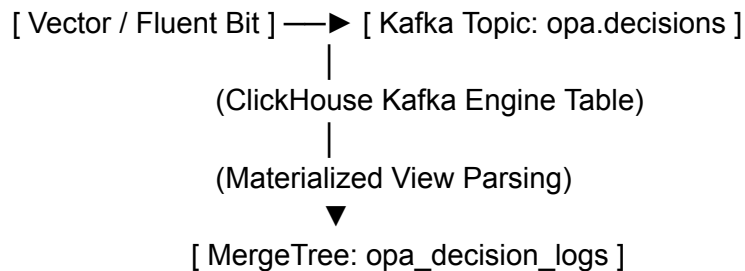
### Key Optimization Mechanics:

- **Partitioning & Sorting Key:** Partitioning by `toYYYYMM` prevents over-partitioning. Sorting primary keys starting with `event_time` and `allowed` optimizes fast time-range scans for denials (`allowed = 0`).

- **LowCardinality Encoding:** Replaces repeated string values (e.g., `tool_name`, `cluster_id`) with dictionary keys, reducing memory overhead by up to 90%.
- **Bloom Filter Skip Indexes:** `bloom_filter` on `spiffe_id` allows ClickHouse to skip data blocks instantly during forensic searches for a single agent across billions of rows.

## 2. High-Throughput Ingestion Architecture (Kafka + Materialized View)

To handle ingestion spikes without backpressuring Vector or OPA, route logs through Apache Kafka before ClickHouse processes them using an engine pipeline.



### Step A: Create Kafka Consumer Engine

```

SQL
CREATE TABLE security_audit.kafka_opa_ingress
(
    raw_payload String
)
ENGINE = Kafka
SETTINGS
    kafka_broker_list = 'kafka-cluster.swarm.internal:9092',
    kafka_topic_list = 'opa.decisions',
    kafka_group_name = 'clickhouse_opa_ingest',
    kafka_format = 'RawBLOB';
  
```

### Step B: Materialized View for Streaming Extraction

The Materialized View triggers on every Kafka batch, extracts the JSON fields in memory, and writes directly into the target `opa_decision_logs` table:

```

SQL
CREATE MATERIALIZED VIEW security_audit.mv_kafka_to_opa_logs
TO security_audit.opa_decision_logs AS
SELECT
    parseDateTime64BestEffortOrNull(JSONExtractString(raw_payload, 'event_timestamp')) AS
    event_time,
  
```

```

    toUUIDOrDefault(JSONExtractString(raw_payload, 'decision_id'), generateUUIDv4()) AS
decision_id,
    JSONExtractString(raw_payload, 'spiffe_id') AS spiffe_id,
    JSONExtractString(raw_payload, 'trust_domain') AS trust_domain,
    toUInt8(JSONExtractBool(raw_payload, 'allowed')) AS allowed,
    JSONExtractString(raw_payload, 'tool_name') AS tool_name,
    JSONExtractString(raw_payload, 'action') AS action,
    JSONExtractString(raw_payload, 'target_resource') AS target_resource,
    toUInt8(JSONExtractInt(raw_payload, 'execution_depth')) AS execution_depth,
    JSONExtractString(raw_payload, 'cluster_id') AS cluster_id,
    JSONExtractString(raw_payload, 'node_id') AS node_id,
    CAST(JSONExtract(raw_payload, 'parameters', 'Map(String, String)'), 'Map(String, String)')
AS parameters,
    raw_payload AS raw_input
FROM security_audit.kafka_opa_ingress;

```

### 3. Pre-Aggregated Rollups for Real-Time Dashboards

Executing `COUNT(*)` over billions of raw rows for continuous SIEM monitoring adds system load. Use a `SummingMergeTree` Materialized View to maintain 1-minute rollup counts automatically:

```

SQL
-- Rollup Target Table
CREATE TABLE security_audit.opa_denials_1m
(
    window_time DateTime,
    spiffe_id String,
    tool_name LowCardinality(String),
    denial_count SimpleAggregateFunction(sum, UInt64)
)
ENGINE = SummingMergeTree()
ORDER BY (window_time, spiffe_id, tool_name);

-- Rollup Materialized View
CREATE MATERIALIZED VIEW security_audit.mv_opa_denials_1m
TO security_audit.opa_denials_1m AS
SELECT
    toStartOfMinute(event_time) AS window_time,
    spiffe_id,
    tool_name,
    count() AS denial_count
FROM security_audit.opa_decision_logs

```

```
WHERE allowed = 0
GROUP BY window_time, spiffe_id, tool_name;
```

## 4. High-Performance Security Analytical Queries

### Query A: Detect Rogue Agent Denial Bursts (High-Frequency Attack Detection)

Find agents accumulating more than 10 policy denials in a 5-minute rolling window:

```
SQL
SELECT
    toStartOfInterval(event_time, INTERVAL 5 MINUTE) AS time_window,
    spiffe_id,
    tool_name,
    count() AS total_denials,
    groupUniqArray(target_resource) AS attempted_resources
FROM security_audit.opa_decision_logs
WHERE
    allowed = 0
    AND event_time >= now() - INTERVAL 1 HOUR
GROUP BY time_window, spiffe_id, tool_name
HAVING total_denials > 10
ORDER BY total_denials DESC;
```

### Query B: Identify Deep Delegation Chains (Runaway Loop Detection)

Query execution depth spikes across agent calls to identify active sub-agent cascades before they cause rate-limit exhaustion:

```
SQL
SELECT
    spiffe_id,
    max(execution_depth) AS max_depth,
    count() AS total_requests,
    quantile(0.99)(execution_depth) AS p99_depth
FROM security_audit.opa_decision_logs
WHERE event_time >= now() - INTERVAL 15 MINUTE
GROUP BY spiffe_id
HAVING max_depth >= 3
ORDER BY max_depth DESC;
```